Project Task Manager - Laravel Implementation

This guide shows how to integrate the Project Task Manager microservice into your existing Laravel Novocib application.

Overview

The Task Manager will be a feature module within your Laravel admin panel, accessible at `/admin/task-manager` with full authentication and authorization.

Step 1: Create Laravel Migrations

1.1 Generate Migration Files

```bash
php artisan make:migration create_projects_table
php artisan make:migration create_tasks_table
php artisan make:migration create_time_entries_table
```

1.2 Projects Migration
(`database/migrations/xxxx_xx_xx_create_projects_table.php`)

```php
<?php

use Illuminate\Database\Migrations\Migration;
use Illuminate\Database\Schema\Blueprint;
use Illuminate\Support\Facades\Schema;

return new class extends Migration
{
    public function up(): void
    {
        Schema::create('projects', function (Blueprint $table) {
            $table->id();
            $table->string('name');
            $table->longText('description')->nullable();
            $table->enum('status', ['active', 'paused', 'completed',
'archived'])->default('active');
            $table->date('start_date')->nullable();
            $table->date('end_date')->nullable();
            $table->foreignId('created_by')->constrained('users');
            $table->timestamps();

            $table->index('status');
            $table->index('created_by');
        });
    }
```

```
    public function down(): void
    {
        Schema::dropIfExists('projects');
    }
};
```

1.3 Tasks Migration (`database/migrations/xxxx_xx_xx_create_tasks_table.php`)

```php
<?php

use Illuminate\Database\Migrations\Migration;
use Illuminate\Database\Schema\Blueprint;
use Illuminate\Support\Facades\Schema;

return new class extends Migration
{
    public function up(): void
    {
        Schema::create('tasks', function (Blueprint $table) {
            $table->id();

$table->foreignId('project_id')->constrained('projects')->cascadeOnDelete();
            $table->string('title');
            $table->longText('description')->nullable();
            $table->enum('status', ['todo', 'in-progress', 'completed',
'blocked'])->default('todo');
            $table->enum('priority', ['low', 'medium', 'high',
'critical'])->default('medium');
            $table->decimal('estimated_hours', 8, 2)->nullable();
            $table->foreignId('assigned_to')->nullable()->constrained('users');
            $table->date('due_date')->nullable();
            $table->timestamps();

            $table->index('project_id');
            $table->index('status');
            $table->index('priority');
            $table->index('assigned_to');
        });
    }

    public function down(): void
    {
        Schema::dropIfExists('tasks');
    }
};
```

1.4 Time Entries Migration

(`database/migrations/xxxx_xx_xx_create_time_entries_table.php`)

```php
<?php

use Illuminate\Database\Migrations\Migration;
use Illuminate\Database\Schema\Blueprint;
use Illuminate\Support\Facades\Schema;

return new class extends Migration
{
    public function up(): void
    {
        Schema::create('time_entries', function (Blueprint $table) {
            $table->id();
            $table->foreignId('task_id')->constrained('tasks')->cascadeOnDelete();

$table->foreignId('project_id')->constrained('projects')->cascadeOnDelete();
            $table->foreignId('user_id')->constrained('users');
            $table->date('date');
            $table->time('start_time')->nullable();
            $table->time('end_time')->nullable();
            $table->decimal('duration_hours', 8, 2);
            $table->longText('notes')->nullable();
            $table->boolean('billable')->default(true);
            $table->timestamps();

            $table->index('project_id');
            $table->index('task_id');
            $table->index('user_id');
            $table->index('date');
        });
    }

    public function down(): void
    {
        Schema::dropIfExists('time_entries');
    }
};
```

1.5 Run Migrations

```bash
php artisan migrate
```

Step 2: Create Models

2.1 Project Model (`app/Models/Project.php`)

```php
<?php

namespace App\Models;

use Illuminate\Database\Eloquent\Model;
use Illuminate\Database\Eloquent\Relations\HasMany;
use Illuminate\Database\Eloquent\Relations\BelongsTo;

class Project extends Model
{
    protected $fillable = [
        'name',
        'description',
        'status',
        'start_date',
        'end_date',
        'created_by',
    ];

    protected $casts = [
        'start_date' => 'date',
        'end_date' => 'date',
        'created_at' => 'datetime',
        'updated_at' => 'datetime',
    ];

    // Relationships
    public function tasks(): HasMany
    {
        return $this->hasMany(Task::class);
    }

    public function timeEntries(): HasMany
    {
        return $this->hasMany(TimeEntry::class);
    }

    public function creator(): BelongsTo
    {
        return $this->belongsTo(User::class, 'created_by');
    }

    // Scopes
    public function scopeActive($query)
    {
        return $query->where('status', 'active');
    }
```

```
    public function scopeCompleted($query)
    {
        return $query->where('status', 'completed');
    }

    // Methods
    public function isActive(): bool
    {
        return $this->status === 'active';
    }

    public function getDaysRemaining(): ?int
    {
        if (!$this->end_date) {
            return null;
        }

        return $this->end_date->diffInDays(now(), false);
    }

    public function getTotalHours(): float
    {
        return $this->timeEntries()->sum('duration_hours');
    }

    public function getCompletedTasksCount(): int
    {
        return $this->tasks()->where('status', 'completed')->count();
    }

    public function getTasksCount(): int
    {
        return $this->tasks()->count();
    }

    public function getProgressPercentage(): int
    {
        $total = $this->getTasksCount();
        if ($total === 0) {
            return 0;
        }

        $completed = $this->getCompletedTasksCount();
        return intval(($completed / $total) * 100);
    }
}
```

2.2 Task Model (`app/Models/Task.php`)

```php
<?php

namespace App\Models;

use Illuminate\Database\Eloquent\Model;
use Illuminate\Database\Eloquent\Relations\HasMany;
use Illuminate\Database\Eloquent\Relations\BelongsTo;

class Task extends Model
{
    protected $fillable = [
        'project_id',
        'title',
        'description',
        'status',
        'priority',
        'estimated_hours',
        'assigned_to',
        'due_date',
    ];

    protected $casts = [
        'due_date' => 'date',
        'created_at' => 'datetime',
        'updated_at' => 'datetime',
    ];

    // Relationships
    public function project(): BelongsTo
    {
        return $this->belongsTo(Project::class);
    }

    public function assignee(): BelongsTo
    {
        return $this->belongsTo(User::class, 'assigned_to');
    }

    public function timeEntries(): HasMany
    {
        return $this->hasMany(TimeEntry::class);
    }

    // Scopes
    public function scopeByStatus($query, string $status)
    {
        return $query->where('status', $status);
    }
```

```
    public function scopeByPriority($query, string $priority)
    {
        return $query->where('priority', $priority);
    }

    public function scopeOverdue($query)
    {
        return $query->where('due_date', '<', now()->toDateString())
                     ->where('status', '!=', 'completed');
    }

    // Methods
    public function isCompleted(): bool
    {
        return $this->status === 'completed';
    }

    public function isOverdue(): bool
    {
        return $this->due_date && $this->due_date->isPast() &&
!$this->isCompleted();
    }

    public function getPriorityColor(): string
    {
        return match($this->priority) {
            'critical' => 'danger',
            'high' => 'warning',
            'medium' => 'info',
            'low' => 'secondary',
            default => 'secondary',
        };
    }

    public function getTotalHours(): float
    {
        return $this->timeEntries()->sum('duration_hours');
    }

    public function getRemainingHours(): ?float
    {
        if (!$this->estimated_hours) {
            return null;
        }

        return $this->estimated_hours - $this->getTotalHours();
    }

    public function getProgressPercentage(): int
    {
```

```
        if (!$this->estimated_hours || $this->estimated_hours == 0) {
            return $this->isCompleted() ? 100 : 0;
        }

        $spent = $this->getTotalHours();
        return intval(($spent / $this->estimated_hours) * 100);
    }
}
```

2.3 TimeEntry Model (`app/Models/TimeEntry.php`)

```php
<?php

namespace App\Models;

use Illuminate\Database\Eloquent\Model;
use Illuminate\Database\Eloquent\Relations\BelongsTo;

class TimeEntry extends Model
{
    protected $table = 'time_entries';

    protected $fillable = [
        'task_id',
        'project_id',
        'user_id',
        'date',
        'start_time',
        'end_time',
        'duration_hours',
        'notes',
        'billable',
    ];

    protected $casts = [
        'date' => 'date',
        'billable' => 'boolean',
        'created_at' => 'datetime',
        'updated_at' => 'datetime',
    ];

    // Relationships
    public function task(): BelongsTo
    {
        return $this->belongsTo(Task::class);
    }

    public function project(): BelongsTo
```

```
    {
        return $this->belongsTo(Project::class);
    }

    public function user(): BelongsTo
    {
        return $this->belongsTo(User::class);
    }

    // Scopes
    public function scopeByProject($query, int $projectId)
    {
        return $query->where('project_id', $projectId);
    }

    public function scopeByUser($query, int $userId)
    {
        return $query->where('user_id', $userId);
    }

    public function scopeBillable($query)
    {
        return $query->where('billable', true);
    }

    public function scopeDateRange($query, $startDate, $endDate)
    {
        return $query->whereBetween('date', [$startDate, $endDate]);
    }

    // Methods
    public function getFormattedDuration(): string
    {
        $hours = intval($this->duration_hours);
        $minutes = intval(($this->duration_hours - $hours) * 60);

        return "{$hours}h {$minutes}m";
    }

    public function getTimeRange(): string
    {
        if (!$this->start_time || !$this->end_time) {
            return $this->getFormattedDuration();
        }

        return "{$this->start_time} - {$this->end_time}";
    }
}
```

Step 3: Create Controllers

3.1 ProjectController
(`app/Http/Controllers/Admin/TaskManager/ProjectController.php`)

```php
<?php

namespace App\Http\Controllers\Admin\TaskManager;

use App\Http\Controllers\Controller;
use App\Models\Project;
use App\Http\Requests\ProjectRequest;
use Illuminate\View\View;
use Illuminate\Http\RedirectResponse;

class ProjectController extends Controller
{
    public function index(): View
    {
        $projects = Project::with('creator')
                           ->orderBy('created_at', 'desc')
                           ->paginate(20);

        return view('admin.task-manager.projects.index', compact('projects'));
    }

    public function create(): View
    {
        return view('admin.task-manager.projects.create');
    }

    public function store(ProjectRequest $request): RedirectResponse
    {
        $project = Project::create([
            ...$request->validated(),
            'created_by' => auth()->id(),
        ]);

        return redirect()
                ->route('admin.projects.show', $project)
                ->with('success', 'Project created successfully.');
    }

    public function show(Project $project): View
    {
        $tasks = $project->tasks()
                         ->orderBy('priority', 'desc')
                         ->orderBy('due_date', 'asc')
                         ->get();
```

```
        $totalHours = $project->getTotalHours();
        $progress = $project->getProgressPercentage();

        return view('admin.task-manager.projects.show', compact('project', 'tasks',
'totalHours', 'progress'));
    }

    public function edit(Project $project): View
    {
        return view('admin.task-manager.projects.edit', compact('project'));
    }

    public function update(ProjectRequest $request, Project $project):
RedirectResponse
    {
        $project->update($request->validated());

        return redirect()
                ->route('admin.projects.show', $project)
                ->with('success', 'Project updated successfully.');
    }

    public function destroy(Project $project): RedirectResponse
    {
        $project->delete();

        return redirect()
                ->route('admin.projects.index')
                ->with('success', 'Project deleted successfully.');
    }
}
```

3.2 TaskController
(`app/Http/Controllers/Admin/TaskManager/TaskController.php`)

```php
<?php

namespace App\Http\Controllers\Admin\TaskManager;

use App\Http\Controllers\Controller;
use App\Models\Task;
use App\Models\Project;
use App\Http\Requests\TaskRequest;
use Illuminate\View\View;
use Illuminate\Http\RedirectResponse;

class TaskController extends Controller
```

```
{
    public function create(Project $project): View
    {
        $users = \App\Models\User::all();
        return view('admin.task-manager.tasks.create', compact('project',
'users'));
    }

    public function store(TaskRequest $request, Project $project): RedirectResponse
    {
        $task = $project->tasks()->create($request->validated());

        return redirect()
                ->route('admin.projects.show', $project)
                ->with('success', 'Task created successfully.');
    }

    public function edit(Task $task): View
    {
        $project = $task->project;
        $users = \App\Models\User::all();

        return view('admin.task-manager.tasks.edit', compact('task', 'project',
'users'));
    }

    public function update(TaskRequest $request, Task $task): RedirectResponse
    {
        $task->update($request->validated());

        return redirect()
                ->route('admin.projects.show', $task->project)
                ->with('success', 'Task updated successfully.');
    }

    public function destroy(Task $task): RedirectResponse
    {
        $project = $task->project;
        $task->delete();

        return redirect()
                ->route('admin.projects.show', $project)
                ->with('success', 'Task deleted successfully.');
    }

    public function updateStatus(Task $task): RedirectResponse
    {
        $task->update(['status' => request('status')]);

        return redirect()
```

```
                ->back()
                ->with('success', 'Task status updated.');
    }
}
```

3.3 TimeEntryController
(`app/Http/Controllers/Admin/TaskManager/TimeEntryController.php`)

```php
<?php

namespace App\Http\Controllers\Admin\TaskManager;

use App\Http\Controllers\Controller;
use App\Models\TimeEntry;
use App\Models\Task;
use App\Models\Project;
use App\Http\Requests\TimeEntryRequest;
use Illuminate\View\View;
use Illuminate\Http\RedirectResponse;
use Illuminate\Http\JsonResponse;

class TimeEntryController extends Controller
{
    public function create(Task $task): View
    {
        $project = $task->project;
        return view('admin.task-manager.time-entries.create', compact('task',
'project'));
    }

    public function store(TimeEntryRequest $request, Task $task): RedirectResponse
    {
        // Calculate duration if start/end times provided
        $duration = $request->duration_hours;

        if (!$duration && $request->start_time && $request->end_time) {
            $start = \Carbon\Carbon::createFromFormat('H:i', $request->start_time);
            $end = \Carbon\Carbon::createFromFormat('H:i', $request->end_time);
            $duration = $start->diffInMinutes($end) / 60;
        }

        $task->timeEntries()->create([
            ...$request->validated(),
            'duration_hours' => $duration,
            'project_id' => $task->project_id,
            'user_id' => auth()->id(),
        ]);
```

```
        return redirect()
                ->route('admin.projects.show', $task->project)
                ->with('success', 'Time logged successfully.');
    }

    public function edit(TimeEntry $timeEntry): View
    {
        $task = $timeEntry->task;
        $project = $task->project;

        return view('admin.task-manager.time-entries.edit', compact('timeEntry',
'task', 'project'));
    }

    public function update(TimeEntryRequest $request, TimeEntry $timeEntry):
RedirectResponse
    {
        // Recalculate duration if start/end times provided
        $duration = $request->duration_hours;

        if ($request->start_time && $request->end_time) {
            $start = \Carbon\Carbon::createFromFormat('H:i', $request->start_time);
            $end = \Carbon\Carbon::createFromFormat('H:i', $request->end_time);
            $duration = $start->diffInMinutes($end) / 60;
        }

        $timeEntry->update([
            ...$request->validated(),
            'duration_hours' => $duration,
        ]);

        return redirect()
                ->route('admin.projects.show', $timeEntry->project)
                ->with('success', 'Time entry updated.');
    }

    public function destroy(TimeEntry $timeEntry): RedirectResponse
    {
        $project = $timeEntry->project;
        $timeEntry->delete();

        return redirect()
                ->route('admin.projects.show', $project)
                ->with('success', 'Time entry deleted.');
    }

    public function report(Project $project): View
    {
        $startDate = request('start_date', now()->startOfMonth());
        $endDate = request('end_date', now()->endOfMonth());
```

```
        $entries = $project->timeEntries()
                          ->whereBetween('date', [$startDate, $endDate])
                          ->orderBy('date', 'desc')
                          ->get();

        $totalHours = $entries->sum('duration_hours');
        $byTask = $entries->groupBy('task_id');

        return view('admin.task-manager.reports.project', compact(
            'project',
            'entries',
            'totalHours',
            'byTask',
            'startDate',
            'endDate'
        ));
    }

    public function apiLog(TimeEntryRequest $request): JsonResponse
    {
        $task = Task::findOrFail($request->task_id);

        $duration = $request->duration_hours;
        if (!$duration && $request->start_time && $request->end_time) {
            $start = \Carbon\Carbon::createFromFormat('H:i', $request->start_time);
            $end = \Carbon\Carbon::createFromFormat('H:i', $request->end_time);
            $duration = $start->diffInMinutes($end) / 60;
        }

        $entry = $task->timeEntries()->create([
            ...$request->validated(),
            'duration_hours' => $duration,
            'project_id' => $task->project_id,
            'user_id' => auth()->id(),
        ]);

        return response()->json([
            'success' => true,
            'entry' => $entry,
            'message' => 'Time logged successfully',
        ]);
    }
}
```

Step 4: Create Form Requests

4.1 ProjectRequest (`app/Http/Requests/ProjectRequest.php`)

```php
<?php

namespace App\Http\Requests;

use Illuminate\Foundation\Http\FormRequest;

class ProjectRequest extends FormRequest
{
    public function authorize(): bool
    {
        return auth()->check() && auth()->user()->isAdmin();
    }

    public function rules(): array
    {
        return [
            'name' => 'required|string|max:255',
            'description' => 'nullable|string|max:5000',
            'status' => 'required|in:active,paused,completed,archived',
            'start_date' => 'nullable|date',
            'end_date' => 'nullable|date|after_or_equal:start_date',
        ];
    }

    public function messages(): array
    {
        return [
            'name.required' => 'Project name is required',
            'status.required' => 'Project status is required',
            'end_date.after_or_equal' => 'End date must be after or equal to start
date',
        ];
    }
}
```

4.2 TaskRequest (`app/Http/Requests/TaskRequest.php`)

```php
<?php

namespace App\Http\Requests;

use Illuminate\Foundation\Http\FormRequest;

class TaskRequest extends FormRequest
{
    public function authorize(): bool
    {
```

```
        return auth()->check() && auth()->user()->isAdmin();
    }

    public function rules(): array
    {
        return [
            'title' => 'required|string|max:255',
            'description' => 'nullable|string|max:5000',
            'status' => 'required|in:todo,in-progress,completed,blocked',
            'priority' => 'required|in:low,medium,high,critical',
            'estimated_hours' => 'nullable|numeric|min:0|max:999.99',
            'assigned_to' => 'nullable|exists:users,id',
            'due_date' => 'nullable|date',
        ];
    }

    public function messages(): array
    {
        return [
            'title.required' => 'Task title is required',
            'status.required' => 'Task status is required',
            'priority.required' => 'Task priority is required',
        ];
    }
}
```

4.3 TimeEntryRequest (`app/Http/Requests/TimeEntryRequest.php`)

```php
<?php

namespace App\Http\Requests;

use Illuminate\Foundation\Http\FormRequest;

class TimeEntryRequest extends FormRequest
{
    public function authorize(): bool
    {
        return auth()->check() && auth()->user()->isAdmin();
    }

    public function rules(): array
    {
        return [
            'task_id' => 'required|exists:tasks,id',
            'date' => 'required|date',
            'start_time' => 'nullable|date_format:H:i',
            'end_time' => 'nullable|date_format:H:i|after:start_time',
```

```
            'duration_hours' => 'nullable|numeric|min:0.25|max:24',
            'notes' => 'nullable|string|max:5000',
            'billable' => 'required|boolean',
        ];
    }

    public function messages(): array
    {
        return [
            'task_id.required' => 'Task is required',
            'date.required' => 'Date is required',
            'end_time.after' => 'End time must be after start time',
        ];
    }
}
```

Step 5: Create Routes

5.1 Task Manager Routes (`routes/task-manager.php`)

Create a new file:

```php
<?php

use Illuminate\Support\Facades\Route;
use App\Http\Controllers\Admin\TaskManager\ProjectController;
use App\Http\Controllers\Admin\TaskManager\TaskController;
use App\Http\Controllers\Admin\TaskManager\TimeEntryController;

Route::middleware(['auth', 'admin'])->prefix('admin/task-manager')->group(function
() {
    // Projects
    Route::resource('projects', ProjectController::class);

    // Tasks
    Route::prefix('projects/{project}')->group(function () {
        Route::resource('tasks', TaskController::class)->only(['create', 'store',
'edit', 'update', 'destroy']);
        Route::patch('tasks/{task}/status', [TaskController::class,
'updateStatus'])->name('tasks.update-status');
    });

    // Time Entries
    Route::prefix('tasks/{task}')->group(function () {
        Route::resource('time-entries', TimeEntryController::class)
              ->only(['create', 'store', 'edit', 'update', 'destroy']);
    });
```

```
    // Reports
    Route::get('projects/{project}/report', [TimeEntryController::class, 'report'])
         ->name('projects.report');

    // API endpoints for AJAX
    Route::post('api/time-entries/log', [TimeEntryController::class, 'apiLog'])
         ->name('api.time-entries.log');
});
```

5.2 Register Routes in `routes/web.php`

Add to your main web routes:

```php
// Task Manager routes
include base_path('routes/task-manager.php');
```

Or better, register it in `app/Providers/RouteServiceProvider.php`:

```php
protected function mapAdminRoutes()
{
    Route::middleware(['web', 'auth'])
        ->namespace($this->namespace)
        ->group(base_path('routes/admin.php'));

    Route::middleware(['web', 'auth'])
        ->namespace($this->namespace)
        ->group(base_path('routes/task-manager.php'));
}
```

Step 6: Create Views

6.1 Projects Index
(`resources/views/admin/task-manager/projects/index.blade.php`)

```blade
@extends('layouts.admin')

@section('title', 'Projects')

@section('content')
<div class="container-fluid">
    <div class="row mb-4">
        <div class="col-md-8">
            <h1>Projects</h1>
        </div>
```

```
        <div class="col-md-4 text-end">
            <a href="{{ route('admin.projects.create') }}" class="btn btn-primary">
                <i class="fas fa-plus"></i> New Project
            </a>
        </div>
    </div>

    @if ($message = Session::get('success'))
        <div class="alert alert-success alert-dismissible fade show" role="alert">
            {{ $message }}
            <button type="button" class="btn-close"
data-bs-dismiss="alert"></button>
        </div>
    @endif

    <div class="card">
        <div class="card-body">
            <table class="table table-hover">
                <thead>
                    <tr>
                        <th>Name</th>
                        <th>Status</th>
                        <th>Tasks</th>
                        <th>Progress</th>
                        <th>Hours</th>
                        <th>Duration</th>
                        <th>Actions</th>
                    </tr>
                </thead>
                <tbody>
                    @forelse($projects as $project)
                    <tr>
                        <td>
                            <a href="{{ route('admin.projects.show', $project) }}">
                                {{ $project->name }}
                            </a>
                        </td>
                        <td>
                            <span class="badge bg-{{ $project->status === 'active'
? 'success' : 'secondary' }}">
                                {{ ucfirst($project->status) }}
                            </span>
                        </td>
                        <td>
                            {{ $project->getCompletedTasksCount() }}/{{
$project->getTasksCount() }}
                        </td>
                        <td>
                            <div class="progress" style="height: 20px;">
                                <div class="progress-bar" role="progressbar"
```

```
                                    style="width: {{
$project->getProgressPercentage() }}%"
                                    aria-valuenow="{{
$project->getProgressPercentage() }}"
                                    aria-valuemin="0" aria-valuemax="100">
                                   {{ $project->getProgressPercentage() }}%
                                </div>
                            </div>
                        </td>
                        <td>
                            {{ number_format($project->getTotalHours(), 1) }}h
                        </td>
                        <td>
                            @if ($project->start_date && $project->end_date)
                                {{ $project->start_date->format('M d') }} - {{
$project->end_date->format('M d, Y') }}
                                @if ($project->getDaysRemaining() > 0)
                                    <br><small class="text-success">{{
$project->getDaysRemaining() }} days left</small>
                                @elseif ($project->getDaysRemaining() === 0)
                                    <br><small class="text-warning">Due
today</small>
                                @else
                                    <br><small class="text-danger">{{
abs($project->getDaysRemaining()) }} days overdue</small>
                                @endif
                            @else
                                N/A
                            @endif
                        </td>
                        <td>
                            <a href="{{ route('admin.projects.show', $project) }}"
class="btn btn-sm btn-info">
                                <i class="fas fa-eye"></i>
                            </a>
                            <a href="{{ route('admin.projects.edit', $project) }}"
class="btn btn-sm btn-warning">
                                <i class="fas fa-edit"></i>
                            </a>
                            <form action="{{ route('admin.projects.destroy',
$project) }}" method="POST" style="display:inline;">
                                @csrf @method('DELETE')
                                <button type="submit" class="btn btn-sm btn-danger"
onclick="return confirm('Delete project?')">
                                    <i class="fas fa-trash"></i>
                                </button>
                            </form>
                        </td>
                    </tr>
                    @empty
```

```
                    <tr>
                        <td colspan="7" class="text-center text-muted">No projects
found</td>
                    </tr>
                    @endforelse
                </tbody>
            </table>
        </div>
    </div>

    <div class="d-flex justify-content-center mt-4">
        {{ $projects->links() }}
    </div>
</div>
@endsection
```

6.2 Project Detail
(`resources/views/admin/task-manager/projects/show.blade.php`)

```blade
@extends('layouts.admin')

@section('title', $project->name)

@section('content')
<div class="container-fluid">
    <div class="row mb-4">
        <div class="col-md-8">
            <h1>{{ $project->name }}</h1>
            <p class="text-muted">{{ $project->description }}</p>
        </div>
        <div class="col-md-4 text-end">
            <a href="{{ route('admin.projects.edit', $project) }}" class="btn
btn-warning">
                <i class="fas fa-edit"></i> Edit
            </a>
            <a href="{{ route('admin.projects.report', $project) }}" class="btn
btn-info">
                <i class="fas fa-chart-bar"></i> Report
            </a>
            <a href="{{ route('admin.projects.index') }}" class="btn
btn-secondary">Back</a>
        </div>
    </div>

    <!-- Metrics Row -->
    <div class="row mb-4">
        <div class="col-md-3">
            <div class="card">
```

```
                <div class="card-body">
                    <h6 class="card-title">Progress</h6>
                    <h3>{{ $progress }}%</h3>
                    <div class="progress">
                        <div class="progress-bar" style="width: {{ $progress
}}%"></div>
                    </div>
                </div>
            </div>
        </div>
        <div class="col-md-3">
            <div class="card">
                <div class="card-body">
                    <h6 class="card-title">Tasks</h6>
                    <h3>{{ $project->getCompletedTasksCount() }}/{{
$project->getTasksCount() }}</h3>
                </div>
            </div>
        </div>
        <div class="col-md-3">
            <div class="card">
                <div class="card-body">
                    <h6 class="card-title">Hours Logged</h6>
                    <h3>{{ number_format($totalHours, 1) }}h</h3>
                </div>
            </div>
        </div>
        <div class="col-md-3">
            <div class="card">
                <div class="card-body">
                    <h6 class="card-title">Status</h6>
                    <span class="badge bg-{{ $project->status === 'active' ?
'success' : 'secondary' }} fs-5">
                        {{ ucfirst($project->status) }}
                    </span>
                </div>
            </div>
        </div>
    </div>

    <!-- Tasks Section -->
    <div class="card">
        <div class="card-header d-flex justify-content-between align-items-center">
            <h5>Tasks</h5>
            <a href="{{ route('admin.tasks.create', $project) }}" class="btn btn-sm
btn-primary">
                <i class="fas fa-plus"></i> New Task
            </a>
        </div>
        <div class="card-body">
```

```
            @if ($tasks->count())
                <table class="table table-hover">
                    <thead>
                        <tr>
                            <th>Title</th>
                            <th>Status</th>
                            <th>Priority</th>
                            <th>Est. Hours</th>
                            <th>Logged Hours</th>
                            <th>Progress</th>
                            <th>Due Date</th>
                            <th>Actions</th>
                        </tr>
                    </thead>
                    <tbody>
                        @foreach($tasks as $task)
                        <tr>
                            <td>{{ $task->title }}</td>
                            <td>
                                <span class="badge bg-info">{{
ucfirst(str_replace('-', ' ', $task->status)) }}</span>
                            </td>
                            <td>
                                <span class="badge bg-{{ $task->getPriorityColor()
}}">
                                    {{ ucfirst($task->priority) }}
                                </span>
                            </td>
                            <td>{{ $task->estimated_hours ?
number_format($task->estimated_hours, 1) . 'h' : 'N/A' }}</td>
                            <td>{{ number_format($task->getTotalHours(), 1)
}}h</td>
                            <td>
                                <div class="progress" style="height: 20px;">
                                    <div class="progress-bar" style="width: {{
$task->getProgressPercentage() }}%">
                                        {{ $task->getProgressPercentage() }}%
                                    </div>
                                </div>
                            </td>
                            <td>
                                @if ($task->due_date)
                                    <span class="badge bg-{{ $task->isOverdue() ?
'danger' : 'secondary' }}">
                                        {{ $task->due_date->format('M d') }}
                                    </span>
                                @else
                                    N/A
                                @endif
                            </td>
```

```
                            <td>
                                <a href="{{ route('admin.time-entries.create',
$task) }}" class="btn btn-sm btn-success"
                                   title="Log time">
                                    <i class="fas fa-clock"></i>
                                </a>
                                <a href="{{ route('admin.tasks.edit', [$project,
$task]) }}" class="btn btn-sm btn-warning">
                                    <i class="fas fa-edit"></i>
                                </a>
                                <form action="{{ route('admin.tasks.destroy',
[$project, $task]) }}" method="POST" style="display:inline;">
                                    @csrf @method('DELETE')
                                    <button type="submit" class="btn btn-sm
btn-danger" onclick="return confirm('Delete task?')">
                                        <i class="fas fa-trash"></i>
                                    </button>
                                </form>
                            </td>
                        </tr>
                        @endforeach
                    </tbody>
                </table>
            @else
                <p class="text-muted">No tasks yet. <a href="{{
route('admin.tasks.create', $project) }}">Create one</a></p>
            @endif
        </div>
    </div>
</div>
@endsection
```

6.3 Time Entry Form
(`resources/views/admin/task-manager/time-entries/create.blade.php`)

```blade
@extends('layouts.admin')

@section('title', 'Log Time - ' . $task->title)

@section('content')
<div class="container-fluid">
    <h1>Log Time for {{ $task->title }}</h1>

    <div class="card">
        <div class="card-body">
            <form action="{{ route('admin.time-entries.store', $task) }}"
method="POST">
                @csrf
```

```
                <div class="mb-3">
                    <label for="date" class="form-label">Date</label>
                    <input type="date" class="form-control @error('date')
is-invalid @enderror"
                           id="date" name="date" value="{{ old('date',
now()->toDateString()) }}" required>
                    @error('date') <span class="invalid-feedback">{{ $message
}}</span> @enderror
                </div>

                <div class="row">
                    <div class="col-md-6">
                        <div class="mb-3">
                            <label for="start_time" class="form-label">Start
Time</label>
                            <input type="time" class="form-control
@error('start_time') is-invalid @enderror"
                                   id="start_time" name="start_time" value="{{
old('start_time') }}">
                            @error('start_time') <span class="invalid-feedback">{{
$message }}</span> @enderror
                        </div>
                    </div>
                    <div class="col-md-6">
                        <div class="mb-3">
                            <label for="end_time" class="form-label">End
Time</label>
                            <input type="time" class="form-control
@error('end_time') is-invalid @enderror"
                                   id="end_time" name="end_time" value="{{
old('end_time') }}">
                            @error('end_time') <span class="invalid-feedback">{{
$message }}</span> @enderror
                        </div>
                    </div>
                </div>

                <div class="mb-3">
                    <label for="duration_hours" class="form-label">Duration
(Hours)</label>
                    <input type="number" step="0.25" class="form-control
@error('duration_hours') is-invalid @enderror"
                           id="duration_hours" name="duration_hours" value="{{
old('duration_hours') }}"
                           placeholder="e.g., 2.5" required>
                    @error('duration_hours') <span class="invalid-feedback">{{
$message }}</span> @enderror
                </div>
```

```
                <div class="mb-3">
                    <label for="notes" class="form-label">Notes</label>
                    <textarea class="form-control @error('notes') is-invalid
@enderror"
                              id="notes" name="notes" rows="4">{{ old('notes')
}}</textarea>
                    @error('notes') <span class="invalid-feedback">{{ $message
}}</span> @enderror
                </div>

                <div class="mb-3 form-check">
                    <input type="checkbox" class="form-check-input" id="billable"
name="billable"
                           value="1" {{ old('billable', true) ? 'checked' : '' }}>
                    <label class="form-check-label" for="billable">
                        Billable
                    </label>
                </div>

                <div class="d-flex gap-2">
                    <button type="submit" class="btn btn-primary">Log Time</button>
                    <a href="{{ route('admin.projects.show', $project) }}"
class="btn btn-secondary">Cancel</a>
                </div>
            </form>
        </div>
    </div>
</div>

<script>
document.addEventListener('DOMContentLoaded', function() {
    const startInput = document.getElementById('start_time');
    const endInput = document.getElementById('end_time');
    const durationInput = document.getElementById('duration_hours');

    function calculateDuration() {
        if (startInput.value && endInput.value) {
            const start = new Date(`2000-01-01 ${startInput.value}`);
            const end = new Date(`2000-01-01 ${endInput.value}`);
            const diff = (end - start) / (1000 * 60 * 60);
            durationInput.value = diff.toFixed(2);
        }
    }

    startInput?.addEventListener('change', calculateDuration);
    endInput?.addEventListener('change', calculateDuration);
});
</script>
@endsection
```

6.4 Reports View
(`resources/views/admin/task-manager/reports/project.blade.php`)

```blade
@extends('layouts.admin')

@section('title', 'Report - ' . $project->name)

@section('content')
<div class="container-fluid">
    <h1>Time Report - {{ $project->name }}</h1>

    <div class="card mb-4">
        <div class="card-body">
            <form action="{{ route('admin.projects.report', $project) }}"
method="GET" class="row g-3">
                <div class="col-md-4">
                    <label for="start_date" class="form-label">Start Date</label>
                    <input type="date" class="form-control" id="start_date"
name="start_date"
                           value="{{ $startDate->toDateString() }}">
                </div>
                <div class="col-md-4">
                    <label for="end_date" class="form-label">End Date</label>
                    <input type="date" class="form-control" id="end_date"
name="end_date"
                           value="{{ $endDate->toDateString() }}">
                </div>
                <div class="col-md-4 d-flex align-items-end gap-2">
                    <button type="submit" class="btn btn-primary">Filter</button>
                    <a href="{{ route('admin.projects.report', $project) }}"
class="btn btn-secondary">Reset</a>
                </div>
            </form>
        </div>
    </div>

    <div class="row mb-4">
        <div class="col-md-3">
            <div class="card">
                <div class="card-body">
                    <h6 class="card-title">Total Hours</h6>
                    <h2>{{ number_format($totalHours, 1) }}h</h2>
                </div>
            </div>
        </div>
        <div class="col-md-3">
            <div class="card">
                <div class="card-body">
```

```
                    <h6 class="card-title">Entries</h6>
                    <h2>{{ $entries->count() }}</h2>
                </div>
            </div>
        </div>
        <div class="col-md-3">
            <div class="card">
                <div class="card-body">
                    <h6 class="card-title">Tasks</h6>
                    <h2>{{ $byTask->count() }}</h2>
                </div>
            </div>
        </div>
        <div class="col-md-3">
            <div class="card">
                <div class="card-body">
                    <h6 class="card-title">Avg per Day</h6>
                    <h2>{{ number_format($totalHours /
($endDate->diffInDays($startDate) + 1), 1) }}h</h2>
                </div>
            </div>
        </div>
    </div>

    <div class="card">
        <div class="card-header">
            <h5>Time by Task</h5>
        </div>
        <div class="card-body">
            <table class="table table-hover">
                <thead>
                    <tr>
                        <th>Task</th>
                        <th>Hours</th>
                        <th>Entries</th>
                        <th>Percentage</th>
                    </tr>
                </thead>
                <tbody>
                    @foreach($byTask as $taskId => $taskEntries)
                    @php
                        $task = $taskEntries->first()->task;
                        $hours = $taskEntries->sum('duration_hours');
                    @endphp
                    <tr>
                        <td>{{ $task->title }}</td>
                        <td>{{ number_format($hours, 1) }}h</td>
                        <td>{{ $taskEntries->count() }}</td>
                        <td>{{ number_format(($hours / $totalHours) * 100, 1)
}}%</td>
```

```
                    </tr>
                    @endforeach
                </tbody>
            </table>
        </div>
    </div>
</div>
@endsection
```

Step 7: Add Admin Middleware

Update your `AdminAuthenticate` middleware
(`app/Http/Middleware/AdminAuthenticate.php`):

```php
<?php

namespace App\Http\Middleware;

use Closure;
use Illuminate\Http\Request;

class AdminAuthenticate
{
    public function handle(Request $request, Closure $next)
    {
        $user = auth()->user();

        if (!$user || !($user->role === 'admin' || $user->role === 'super-admin'))
{
            abort(403, 'Unauthorized - Admin access required');
        }

        return $next($request);
    }
}
```

Step 8: Add Navigation Menu Item

Update your admin layout (`resources/views/layouts/admin.blade.php`):

```blade
<nav class="navbar navbar-expand-lg navbar-dark bg-dark">
    <div class="container-fluid">
        <a class="navbar-brand" href="/admin">Admin Panel</a>
        <button class="navbar-toggler" type="button" data-bs-toggle="collapse"
data-bs-target="#navbarNav">
            <span class="navbar-toggler-icon"></span>
```

```
        </button>
        <div class="collapse navbar-collapse" id="navbarNav">
            <ul class="navbar-nav ms-auto">
                <li class="nav-item">
                    <a class="nav-link" href="{{ route('admin.dashboard')
}}">Dashboard</a>
                </li>
                <li class="nav-item">
                    <a class="nav-link" href="{{ route('admin.products.index')
}}">Products</a>
                </li>
                <li class="nav-item">
                    <a class="nav-link" href="{{ route('admin.messages.index')
}}">Messages</a>
                </li>
                <li class="nav-item dropdown">
                    <a class="nav-link dropdown-toggle" href="#"
id="taskManagerNav" role="button" data-bs-toggle="dropdown">
                        <i class="fas fa-tasks"></i> Task Manager
                    </a>
                    <ul class="dropdown-menu" aria-labelledby="taskManagerNav">
                        <li><a class="dropdown-item" href="{{
route('admin.projects.index') }}">Projects</a></li>
                        <li><a class="dropdown-item" href="{{
route('admin.projects.create') }}">New Project</a></li>
                    </ul>
                </li>
                <li class="nav-item">
                    <a class="nav-link" href="{{ route('logout') }}">Logout</a>
                </li>
            </ul>
        </div>
    </div>
</nav>
```

Step 9: Database Seeding (Optional)

Create a seeder (`database/seeders/ProjectSeeder.php`):

```php
<?php

namespace Database\Seeders;

use Illuminate\Database\Seeder;
use App\Models\Project;
use App\Models\Task;

class ProjectSeeder extends Seeder
```

```
{
    public function run(): void
    {
        $user = \App\Models\User::first();

        $project = Project::create([
            'name' => 'Sample Project',
            'description' => 'This is a sample project to test the task manager',
            'status' => 'active',
            'start_date' => now()->toDateString(),
            'end_date' => now()->addMonth()->toDateString(),
            'created_by' => $user->id,
        ]);

        Task::create([
            'project_id' => $project->id,
            'title' => 'Sample Task',
            'description' => 'This is a sample task',
            'status' => 'todo',
            'priority' => 'high',
            'estimated_hours' => 5,
            'due_date' => now()->addWeek()->toDateString(),
        ]);
    }
}
```

Run with:

```bash
php artisan db:seed --class=ProjectSeeder
```

Key Integration Points

In Your Existing Admin Structure

1. **Routes**: Add to your admin routes configuration
2. **Navigation**: Include in admin sidebar/menu
3. **Authentication**: Uses existing `auth` and `admin` middleware
4. **Database**: Separate tables but same database
5. **Models**: Follows Laravel conventions
6. **Controllers**: In `Admin\TaskManager` namespace

Authorization

All routes are protected with:
- `auth` middleware (must be logged in)
- `admin` middleware (must be admin user)

API Endpoints

For AJAX/frontend integration:

```javascript
// Log time via API
fetch('/admin/task-manager/api/time-entries/log', {
    method: 'POST',
    headers: {
        'Content-Type': 'application/json',
        'X-CSRF-TOKEN': document.querySelector('meta[name="csrf-token"]').content
    },
    body: JSON.stringify({
        task_id: 1,
        date: '2024-04-30',
        duration_hours: 2.5,
        notes: 'Completed feature',
        billable: true
    })
})
.then(r => r.json())
.then(data => console.log(data));
```

Summary

The Project Task Manager integrates seamlessly into Laravel:

✅ Uses Laravel migrations and models
✅ Follows MVC architecture
✅ Protected by existing admin auth
✅ Blade templating for views
✅ Form request validation
✅ Eloquent relationships
✅ Route resource conventions
✅ API endpoints for AJAX
✅ Reporting capabilities
✅ Real-time duration calculation

The implementation is production-ready and follows Laravel best practices.